Proyecto 2

Emily Ariana Gongora Giron #243008

Link a repo; https://github.com/EmilyGG24308/PROYECTO-2-TEORIA

Qué hace el programa

El programa revisa si una frase en inglés pertenece al lenguaje de una gramática libre de contexto. Para hacerlo usa CYK, que necesita una gramática en Forma Normal de Chomsky (CNF). Si la gramática no está en esa forma, primero la simplifica y la convierte.

También permite cargar gramáticas desde archivos y analizar expresiones como (id*id)+id. Muestra la conversión, la tabla CYK, la respuesta SÍ o NO, el tiempo de validación y un árbol cuando acepta la entrada. La rúbrica asigna 7 puntos a simplificación, 7 a CYK y 1 a documentación. Sin documentación no hay derecho a nota.

Cómo funciona

Una gramática libre de contexto tiene terminales, no terminales, producciones y un símbolo inicial. Los terminales son las palabras o símbolos de la entrada. Los no terminales agrupan partes de la estructura. Una cadena pertenece al lenguaje si puede obtenerse desde el símbolo inicial siguiendo las producciones.

En CNF, cada regla produce dos no terminales, como A → B C, o un terminal, como A → a. Se permite inicial → ε si el lenguaje acepta la cadena vacía. En ese caso, el inicial no puede aparecer a la derecha de ninguna producción.

CYK divide la entrada en fragmentos y guarda qué no terminales generan cada uno. Empieza con un token y luego combina fragmentos cada vez más largos. Esta es la programación dinámica: guardar respuestas pequeñas para usarlas después sin calcularlas de nuevo.

Para una cadena de n tokens, la celda T[l][i] representa el fragmento de longitud l que empieza en i. Primero se agregan los A que cumplen A → token. Después se prueban todas las divisiones del fragmento. Si una parte puede ser B, la otra C y existe A → B C, se agrega A. La entrada se acepta si el símbolo inicial aparece en la celda que cubre toda la cadena.

Diseño y datos

Usé Python y una consola para concentrarme en los algoritmos, sin dependencias externas. El enunciado no exige un lenguaje. El lector carga el archivo y crea una Gramatica. El conversor devuelve la CNF y sus etapas. El tokenizador prepara la entrada y AnalizadorCYK la revisa. Las funciones de presentación dan formato a la gramática, la tabla y el árbol. Solo Principal, en main.py, interactúa con el usuario. Los demás módulos devuelven resultados o lanzan ErrorGramatica.

Las producciones se guardan en un diccionario. Cada lado derecho es una tupla de símbolos. Por ejemplo, F → (E) | id se guarda como {"F": [("(", "E", ")"), ("id",)]}. Así, id, NP y Det cuentan como un símbolo cada uno. Epsilon es la tupla vacía. El lector reúne primero todos los lados izquierdos para identificar los no terminales, conserva el orden del archivo y elimina duplicados.

La tabla es triangular. En el código, tabla[l-1][i] guarda un diccionario de no terminales y referencias. Cada referencia registra la producción y el punto de división. Se conserva la primera por cada no terminal y celda,

suficiente para reconstruir un árbol válido. La reconstrucción sigue esas referencias hasta los terminales. Siempre avanza hacia fragmentos más cortos.

Conversión a CNF

Primero se revisa el símbolo inicial. Si es anulable y aparece en un lado derecho, se agrega uno nuevo con una regla como S0 → S. Esto permite conservar la cadena vacía sin incumplir CNF.

Después se calculan los símbolos anulables hasta que el conjunto deja de cambiar. Para cada producción se generan las combinaciones de conservar u omitir sus símbolos anulables. Se descartan los lados derechos vacíos, salvo el del inicial cuando corresponde. Con m posiciones anulables puede haber 2^m combinaciones.

Luego se eliminan las reglas unitarias, como A → B. Para cada no terminal se buscan todos los que alcanza usando solo esas reglas y se copian sus producciones no unitarias. El recorrido controla los símbolos visitados, por lo que también funciona con ciclos.

El siguiente paso elimina símbolos inútiles. Primero quita los que no generan cadenas de terminales. Después calcula cuáles siguen siendo alcanzables desde el inicial. Si el inicial no genera ninguna cadena, el lenguaje es vacío y se rechaza toda entrada.

Los terminales que aparecen en lados derechos de dos o más símbolos se sustituyen por auxiliares TERM_*. Cada terminal comparte un auxiliar. Las producciones de más de dos símbolos se dividen con auxiliares BIN*. Las secuencias finales idénticas pueden reutilizar el mismo auxiliar. GeneradorNombres revisa todos los nombres existentes y agrega sufijos para evitar colisiones.

Al terminar, problemas_cnf comprueba las reglas y la excepción de epsilon. Si algo no cumple, lanza un error. Este orden evita dejar reglas unitarias creadas al eliminar epsilon y símbolos inútiles creados durante la simplificación. Si la gramática ya está en CNF, se usa directamente. La opción --forzar permite recorrer las etapas de todos modos.

En 1.txt, X y Y son anulables. Al quitar epsilon aparecen E → T y T → F. Al eliminar esas unitarias, E recibe las alternativas de T y F. No hay símbolos inútiles. Después se crean auxiliares para los paréntesis, + y *, y se dividen las reglas largas. E no es anulable, así que no necesita un inicial nuevo.

La CNF final tiene la misma estructura que 1-cnf.txt. TERM_PAR_IZQ, TERM_PAR_DER, TERM_MAS y TERM_POR corresponden a L, R, P y M. BIN1, BIN2 y BIN3 corresponden a Z, W y V. El desarrollo completo está en docs/ejecuciones/expresiones_1txt.txt. La gramática de inglés ya está en CNF, por lo que CYK puede usarla directamente. Su ejecución forzada está en docs/ejecuciones/ingles_forzado.txt.

Análisis y tiempo

AnalizadorCYK crea índices para buscar reglas por terminal y por pareja de no terminales. Así evita recorrer todas las producciones en cada celda. Para una entrada vacía comprueba directamente si existe inicial → ε. Si encuentra un token desconocido, deja su celda base vacía y explica el rechazo.

El tokenizador tiene los modos auto, palabras y simbolos. En palabras convierte la entrada a minúsculas, pero no borra puntuación desconocida. En simbolos reconoce id, +, *, (y), incluso sin espacios. Los árboles corresponden a la gramática CNF y pueden mostrar sus auxiliares. En una gramática ambigua se devuelve un árbol, no todos.

La validación se mide con time.perf_counter(). Incluye la inicialización y el llenado de la tabla y la decisión final. Excluye lectura, conversión, índices, tokenización, interacción e impresión. La reconstrucción del árbol se mide aparte.

Con una gramática fija, el tiempo crece como $O(n^3)$: hay $O(n^2)$ celdas y $O(n)$ divisiones por celda. El tamaño de la gramática también afecta el costo. Esta implementación combina los no terminales de las dos celdas y consulta el índice de reglas binarias. La tabla ocupa $O(n^2 \cdot |V|)$, donde $|V|$ es el número de no terminales. Guardar todas las referencias para enumerar árboles puede elevar el espacio a $O(n^3 \cdot |P|)$. Reconstruir un árbol toma $O(n)$. En la conversión, eliminar epsilon puede ser exponencial en los anulables de una producción; las demás etapas son polinomiales.

Ejemplos y pruebas

En inglés se aceptan “she eats a cake with a fork”, “the cat drinks the beer” y “he cooks”. También se acepta “she eats a fork”, porque la gramática revisa estructura, no sentido. “She eats a cake with a fork” da el mismo resultado al convertir las mayúsculas.

Se rechazan “she a cake”, porque falta el verbo; “the cat drinks beer”, porque el sustantivo necesita determinante; y “eats she a cake”, porque la oración debe empezar con NP. “she eats a pizza” contiene un terminal desconocido. “he cooks.” también se rechaza: el punto no se elimina.

Con 1.txt convertido y con 1-cnf.txt se aceptan id, id + id, id * id y (id*id)+id, con o sin espacios. Se rechazan id +, * id, (id + id e id id. La cadena vacía también se rechaza porque E no es anulable.

En la ejecución de “She eats a cake with a fork”, la tabla termina con S en la celda superior. La validación tardó 0,0336 ms y el árbol 0,0132 ms. El árbol separa NP → she y un VP formado por “eats a cake” y el PP “with a fork”. La salida está en docs/ejecuciones/ingles.txt.

Para (id*id)+id, la celda superior contiene E y BIN2. Se acepta porque contiene E. La validación tardó 0,0381 ms y la reconstrucción 0,0133 ms. El árbol divide la expresión en T, que genera (id*id), y X, que genera +id. Dentro del paréntesis, E genera id*id.

El comando python -m unittest discover -v ejecutó 31 pruebas y todas pasaron en unos 3,2 segundos, con Python 3.14.5 en Windows 11. test_lector.py tiene 9 pruebas, test_cnf.py tiene 12 y test_cyk.py tiene 10. Cubren lectura, errores de formato, epsilon, ciclos unitarios, símbolos inútiles, lenguaje vacío, auxiliares, gramáticas ya en CNF, tokenización, ambigüedad y rechazo de gramáticas fuera de CNF. Para cada árbol aceptado se comprueba que sus hojas coincidan con los tokens y que cada nodo use una producción válida. La salida está en docs/ejecuciones/pruebas.txt.

También se comparó CYK con un reconocedor Earley independiente, en tests/utilidades.py, que usa la gramática original y maneja epsilon con la corrección de Aycock y Horspool. Para 1.txt se revisaron las 19 531 cadenas de hasta 6 tokens, de las cuales 15 fueron aceptadas. En gramáticas pequeñas se probaron cadenas de hasta 5, 6 u 8 tokens, según el alfabeto. No hubo diferencias. Al quitar a propósito Y → M F de 1-cnf.txt, la comparación detectó el error en id * id.

Estas pruebas aportan evidencia de que la conversión conserva el lenguaje en los casos revisados. No demuestran equivalencia para todas las cadenas. La justificación general depende de las transformaciones descritas por Hopcroft, Motwani y Ullman.

Los tiempos de herramientas/medir_tiempos.py son medianas de 25 repeticiones en un Intel x86-64, Windows 11 y Python 3.14.5. En inglés, las entradas de 4, 7, 10, 16, 28 y 52 tokens tardaron 0,0067; 0,0174; 0,0349; 0,0949; 0,4060 y 2,2605 ms. En expresiones, 3, 7, 15, 31 y 63 tokens tardaron 0,0053; 0,0253; 0,1409; 0,9127 y 6,6856 ms.

Al pasar de 31 a 63 tokens, el tiempo de expresiones se multiplicó por unos 7,3, cerca del factor 8 esperado al duplicar n en un algoritmo cúbico. En inglés, pasar de 28 a 52 tokens multiplicó el tiempo por unos 5,6. Los valores cambian según la máquina y la ejecución.

Dificultades y límites

El PDF de ejemplo tenía páginas escaneadas y fue necesario leer las imágenes para comparar el desarrollo. También se comprobó que la gramática de inglés ya estaba en CNF. Por eso la conversión se demuestra con 1.txt y las gramáticas pequeñas. Las 31 pruebas pasaron en la primera ejecución completa; la alteración intencional de una regla permitió comprobar que la comparación con Earley sí detectaba diferencias.

El formato de gramáticas exige espacios entre símbolos y reserva e para epsilon. En modo palabras, los terminales deben estar en minúsculas. La eliminación de epsilon puede generar muchas combinaciones y el árbol mostrado conserva los auxiliares de CNF.

Para gramáticas grandes convendría evaluar recorrer las producciones binarias o usar conjuntos de bits. Binarizar antes de eliminar epsilon, como proponen Lange y Leiß, ayudaría a evitar la explosión de combinaciones. Aquí se siguió el orden clásico del curso. Para mostrar todos los árboles habría que guardar más referencias. Para presentar el árbol con símbolos originales habría que aplanar los nodos auxiliares.

Referencias

Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3.ª ed.). Pearson/Addison-Wesley. Cap. 7 (formas normales, CYK).

Sipser, M. (2012). Introduction to the Theory of Computation (3.ª ed.). Cengage Learning. Sec. 2.1 y 7.2.

Kasami, T. (1965). An efficient recognition and syntax-analysis algorithm for context-free languages. Scientific Report AFCRL-65-758, Air Force Cambridge Research Laboratory.

Younger, D. H. (1967). Recognition and parsing of context-free languages in time n^3. Information and Control, 10(2), 189–208.

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). Introduction to Algorithms (3.ª ed.). MIT Press. Cap. 15 (programación dinámica).

Lange, M. y Leiß, H. (2009). To CNF or not to CNF? An efficient yet presentable version of the CYK algorithm. Informatica Didactica, 8.

Earley, J. (1970). An efficient context-free parsing algorithm. Communications of the ACM, 13(2), 94–102.

Aycock, J. y Horspool, R. N. (2002). Practical Earley parsing. The Computer Journal, 45(6), 620–630.

Jurafsky, D. y Martin, J. H. Speech and Language Processing (3.ª ed., borrador en línea). Capítulo de análisis de constituyentes (algoritmo CKY). https://web.stanford.edu/~jurafsky/slp3/